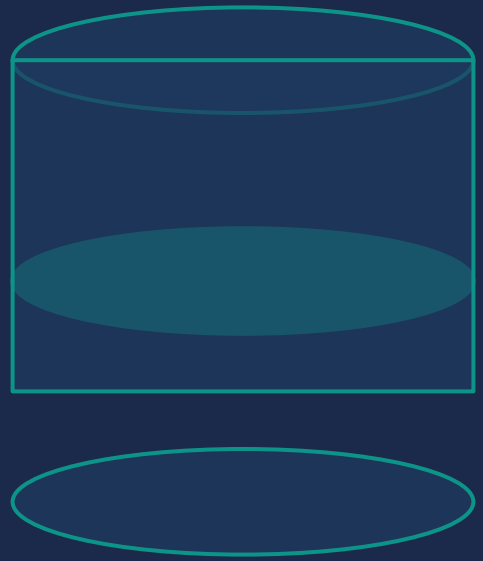
Normalización de Bases de Datos

De la tabla sin forma... a las Formas Normales

1FN **2FN** **3FN**

Paso a paso con un ejercicio práctico completo



¿Qué es la Normalización?

Teoría base antes de empezar

Definición

La normalización es el proceso de organizar una base de datos relacional para:

- Eliminar redundancia de datos
- Evitar anomalías de inserción, borrado y actualización
- Garantizar dependencias lógicas entre datos
- Facilitar el mantenimiento y escalabilidad

Las Formas Normales

1FN Sin grupos repetitivos ni valores múltiples

2FN Sin dependencias parciales a la clave

3FN Sin dependencias transitivas

Cada forma normal incluye a todas las anteriores.
1FN ⊂ 2FN ⊂ 3FN

Conceptos Clave: Dependencias Funcionales

Entender esto es fundamental para normalizar

Dependencia Funcional

A → B

B depende funcionalmente de A cuando, para cada valor de A, existe exactamente un valor de B.

Ej: DNI → Nombre
(El nombre depende del DNI)

Dependencia Parcial

PK → B

B depende solo de UNA PARTE de la clave primaria compuesta, no de toda.

Ej: (CursoID, AlumnoID) → NombreCurso
Solo depende de CursoID

Dependencia Transitiva

A→B→C

C depende de A a través de B: A → B y B → C, por lo tanto A → C.

Ej: AlumnoID → CiudadID → NombreCiudad

⚠ ***Las dependencias parciales y transitivas son los 'problemas' que la normalización elimina.***

El Ejercicio: Tabla de Pedidos

Punto de partida — tabla sin normalizar

Tenemos esta tabla que registra pedidos de una librería:

Pedido ID	FechaPedido	Cliente ID	NombreCliente	Ciudad	CodigoPostal	LibroID	TituloLibro	Autor	Cantidad	Precio
P01	10/03/2024	C01	Ana García	Mendoza	5500	L01	El Principito	St-Exupéry	2	$1200
P01	10/03/2024	C01	Ana García	Mendoza	5500	L02	1984	Orwell	1	$980
P02	11/03/2024	C02	Juan Pérez	Córdoba	5000	L01	El Principito	St-Exupéry	3	$1200
P03	12/03/2024	C01	Ana García	Mendoza	5500	L03	Don Quijote	Cervantes	1	$1500

Problemas que vemos en esta tabla:

Redundancia

"Ana García" y "Mendoza" se repiten en múltiples filas

Anomalía actualización

Si Ana se muda, hay que cambiar MUCHAS filas

Grupos repetitivos

Varios libros en el mismo pedido duplican todo lo demás

Primera Forma Normal (1FN)

Teoría: los requisitos que debe cumplir

1FN

1

Valores atómicos

Todos los atributos deben contener valores atómicos (simples indivisibles). No puede haber listas, ni conjuntos.

✓ **Ciudad: 'Mendoza'**

✗ **Teléfonos: '261-111, 261-222'**

2

Sin grupos repetitivos

No puede haber columnas del tipo Libro1, Libro2, Libro3... que repitan el mismo tipo de dato.

✓ **Una fila por libro-pedido**

✗ **Columnas: Libro1, Libro2, Libro3**

Aplicando la 1FN al Ejercicio

¿Nuestra tabla ya cumple 1FN?

Revisamos la tabla original:

✓ ¿Valores atómicos? → SÍ, cada celda tiene un solo valor

✓ ¿Sin grupos repetitivos en columnas? → SÍ, no hay Libro1, Libro2...

Tabla en 1FN

PedidoID	FechaPedido	ClienteID	NombreCliente	Ciudad	CP	LibroID	TituloLibro	Autor	Cantidad	Precio
P01	10/03/2024	C01	Ana García	Mendoza	5500	L01	El Principito	St-Exupéry	2	$1200
P01	10/03/2024	C01	Ana García	Mendoza	5500	L02	1984	Orwell	1	$980
P02	11/03/2024	C02	Juan Pérez	Córdoba	5000	L01	El Principito	St-Exupéry	3	$1200
P03	12/03/2024	C01	Ana García	Mendoza	5500	L03	Don Quijote	Cervantes	1	$1500

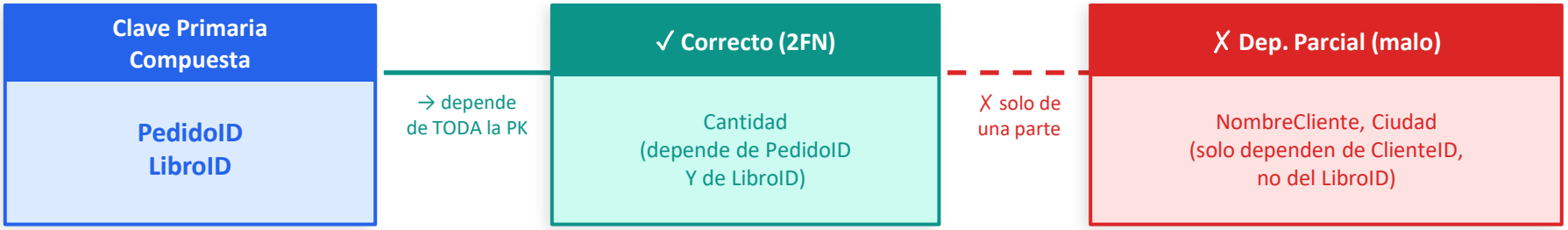
Segunda Forma Normal (2FN)

Teoría: eliminar dependencias parciales

Definición: Una tabla está en 2FN si:

(1) Está en 1FN
(2) **todos los atributos no-clave dependen de TODA la clave primaria** (no solo de una parte - no hay dependencias parciales).

Definir clave primaria: la clave debe ser (PedidoID + LibroID) para identificar cada fila únicamente



Solución: Descomponer la tabla. Los atributos con dependencia parcial se llevan a una tabla nueva, junto con la parte de la clave de la que dependen.

2FN: Analizando las Dependencias del Ejercicio

¿Qué depende de qué?

Clave Primaria: (PedidoID, LibroID) — Analizamos CADA atributo:

Atributo	¿De qué depende?	Tipo	Acción
FechaPedido	PedidoID (parte de PK)	⚠ Parcial	→ Tabla Pedidos
ClienteID	PedidoID (parte de PK)	⚠ Parcial	→ Tabla Pedidos
NombreCliente	ClienteID (no-clave)	⚠ Parcial/Transitiva	→ Tabla Clientes
Ciudad	ClienteID (no-clave)	⚠ Parcial/Transitiva	→ Tabla Clientes
CodigoPostal	ClienteID (no-clave)	⚠ Parcial/Transitiva	→ Tabla Clientes
TituloLibro	LibroID (parte de PK)	⚠ Parcial	→ Tabla Libros
Autor	LibroID (parte de PK)	⚠ Parcial	→ Tabla Libros
Precio	LibroID (parte de PK)	⚠ Parcial	→ Tabla Libros
Cantidad	PedidoID + LibroID (toda la PK)	✓ Total	Queda aquí

→ Solo Cantidad tiene dependencia TOTAL. Los demás deben moverse a otras tablas.

Resultado de Aplicar la 2FN

Descomponemos en 4 tablas

PEDIDO_DETALLE

PedidoID	LibroID	Cantidad
P01	L01	2
P01	L02	1
P02	L01	3
P03	L03	1

PEDIDOS

PedidoID	FechaPedido	ClienteID
P01	10/03/2024	C01
P02	11/03/2024	C02
P03	12/03/2024	C01

LIBROS

LibroID	TituloLibro	Autor	Precio
L01	El Principito	St-Exupéry	$1200
L02	1984	Orwell	$980
L03	Don Quijote	Cervantes	$1500

CLIENTES

ClienteID	NombreCliente	Ciudad	CP
C01	Ana García	Mendoza	5500
C02	Juan Pérez	Córdoba	5000

¡La redundancia de datos de pedidos y libros ya se eliminó!

Tercera Forma Normal (3FN)

Teoría: eliminar dependencias transitivas

Definición: Una tabla está en 3FN si:

(1) Ya está en 2FN, Y (2) **ningún atributo no-clave depende de otro atributo no-clave.** *(sin dependencias transitivas)*

Visualizando una dependencia transitiva:



`ClienteID → Ciudad → CodigoPostal` ∴ `ClienteID → CodigoPostal (transitiva!)`

El Problema

Si Ana García se muda a San Juan (CP 5400), hay que actualizar TODOS los registros que la tienen. Si nos olvidamos uno, la BD queda inconsistente.

La Solución

Separar Ciudad y CodigoPostal a su propia tabla con Ciudad como clave. Así, cambiar una ciudad se hace en UN SOLO lugar.

3FN: Analizando la Tabla CLIENTES

¿Hay dependencias transitivas?

Tomamos la tabla CLIENTES que obtuvimos en el paso anterior:

ClienteID 🔑	NombreCliente	Ciudad	CodigoPostal
C01	Ana García	Mendoza	5500
C02	Juan Pérez	Córdoba	5000
C03	María López	Mendoza	5500

Análisis de dependencias en CLIENTES:

`ClienteID → NombreCliente`	Directa ✓	El nombre depende directamente de la clave. OK.
`ClienteID → Ciudad`	Directa ✓	La ciudad depende directamente de la clave. OK.
`ClienteID → Ciudad`	Transitiva ✗	La Ciudad depende de la CP, no directamente del ClienteID.
`CodigoPostal → Ciudad`		Todo CP tiene una ciudad asociada → dependencia entre no-claves.

→ Solución: separar Ciudad/CP en su propia tabla CIUDADES con Ciudad como clave primaria.

Resultado Final: Base de Datos en 3FN

5 tablas normalizadas

PEDIDO_DETALLE

PedidoID	LibroID	Cantidad
P01	L01	2
P01	L02	1
P02	L01	3

PEDIDOS

PedidoID	Fecha	ClienteID
P01	10/03	C01
P02	11/03	C02
P03	12/03	C01

LIBROS

LibroID	Titulo	Autor	Precio
L01	El Principito	St-Exupéry	$1200
L02	1984	Orwell	$980

CLIENTES

ClienteID	Nombre	Ciudad
C01	Ana García	Mendoza
C02	Juan Pérez	Córdoba

CIUDADES

Ciudad	CodigoPostal
Mendoza	5500
Córdoba	5000
San Juan	5400

✓ Sin redundancia ✓ Sin anomalías de actualización ✓ Datos consistentes ✓ Fácil de mantener

Resumen del Proceso de Normalización

1FN — Primera Forma Normal

- Valores atómicos en cada celda
- Sin grupos repetitivos de columnas

2FN — Segunda Forma Normal

- Ya en 1FN
- Definir clave primaria (puede ser compuesta)
- Eliminar dependencias parciales
- Separar atributos que dependen solo de parte de la PK

3FN — Tercera Forma Normal

- Ya en 2FN
- Eliminar dependencias transitivas
- Ningún atributo no-clave depende de otro no-clave

Regla práctica: Los atributos no-clave deben depender de **de toda la clave.**